UNIT IV
Inheritance & Pointers

Solved Debugging Questions, Corrected C++ Programs, Pseudo-Code and Dry Runs

SECTION A – DEBUGGING QUESTIONS

The answers below follow the five debugging problems in the supplied Unit IV question bank. Console output is representative; memory addresses and compiler wording can vary.

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

Bug locations: Lines 12 and 18 use ordinary (non-virtual) inheritance; line 33 accesses ta.idNumber ambiguously.

Defect category: Duplicate base-class subobjects / ambiguous inherited member.

Explanation: TeachingAssistant contains a Student subobject and an Employee subobject. Each of those contains its own Person subobject. Therefore the complete object has two idNumber members. The expression ta.idNumber does not identify which Person subobject is intended.

Correction: Make both intermediate classes inherit Person virtually. The most-derived class TeachingAssistant is responsible for initializing the shared virtual Person base.

```
#include <iostream>

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << '\n';
    }
};

class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int id)
        : Person(id), Student(id), Employee(id) {}
};

int main() {
    TeachingAssistant ta(101);
    std::cout << "TA ID: " << ta.idNumber << '\n';
    return 0;
}
```

Output:

```
Person initialized: 101
TA ID: 101
```

Constructor rule: When a virtual base is present, only the most-derived class initializes that virtual base. The Person(id) initializers inside Student and Employee are ignored when those classes are constructed as part of TeachingAssistant. If Student or Employee is created directly, its own Person initializer is used.

Memory layout (conceptual): TeachingAssistant has one shared Person virtual base, one Student portion, one Employee portion, and its own derived portion. The exact physical layout is implementation-dependent.

Question 2: Missing Base Constructor in Derived Initializer List

Bug location: Line 18 – Sensor(std::string mac, int rate) has no Device(mac) initializer.

Defect category: Required base-class constructor is not invoked.

Explanation: Device declares only Device(std::string). It has no zero-argument constructor. Before the Sensor constructor body begins, C++ must construct its Device base. Since no initializer is provided, the compiler attempts to call Device(), which does not exist. Assigning macAddress inside the constructor body is too late: the base object must already have been constructed.

Corrected code:

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;

public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: "
                  << macAddress << '\n';
    }
};

class Sensor : public Device {
private:
    int samplingRate;

public:
    Sensor(std::string mac, int rate)
        : Device(mac), samplingRate(rate) {}

    void display() const {
        std::cout << "MAC: " << macAddress
                  << ", Rate: " << samplingRate << " Hz\n";
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Constructor sequence: (1) Device base constructor, (2) Sensor data members in declaration order, (3) Sensor constructor body. For a class with member objects, those member objects are constructed after base classes and before the derived constructor body.

Output:

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

Question 3: Base Class Pointer Member Access and Object Slicing

Bug locations: Line 22 calls engageTurbo() through Vehicle*; line 26 copies SportsCar into a Vehicle object.

Defect categories: Static type checking failure and object slicing.

Explanation: vPtr has the static type Vehicle*. The compiler permits only members declared in Vehicle to be called through it, even though the pointer currently stores the address of a SportsCar. Object slicing occurs when a SportsCar is copied into a standalone Vehicle value: only the Vehicle base portion is copied, so turboEnabled is not part of plainVehicle.

Correction: Use dynamic_cast when the base class is polymorphic. Add a virtual destructor for safe deletion through a base pointer. Use a base reference or pointer to refer to the derived object without slicing.

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() = default;

    void honk() const {
        std::cout << "General Horn\n";
    }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;

    SportsCar(int s, bool turbo)
        : Vehicle(s), turboEnabled(turbo) {}

    void engageTurbo() const {
        std::cout << "Turbo Engaged!\n";
    }
};

int main() {
    Vehicle* vPtr = new SportsCar(300, true);

    vPtr->honk();

    SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr);
    if (sportsPtr != nullptr)
        sportsPtr->engageTurbo();

    SportsCar car(250, true);
    Vehicle& vehicleView = car;  // No slicing
    std::cout << "Max speed: " << vehicleView.maxSpeed << '\n';

    delete vPtr;
    return 0;
}
```

Output:

```
General Horn
Turbo Engaged!
Max speed: 250
```

A static_cast<SportsCar*>(vPtr) is also possible when the programmer can guarantee the actual object is SportsCar, but dynamic_cast is safer for checked downcasting. The base class must be polymorphic (for example, have a virtual function) for this dynamic_cast form.

Question 4: Incorrect Return Type in this Pointer Method Chaining

Bug location: Line 11 – setWidth returns WindowBox* and returns this.

Defect category: Pointer/reference return mismatch in fluent method chaining.

Explanation: this is a pointer to the current object (WindowBox*), while *this is the object itself as an lvalue. Dot syntax such as .setHeight() requires an object or reference, not a pointer. Returning WindowBox& and return *this enables chaining.

```
#include <iostream>

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width
                  << " x " << height << '\n';
    }
};

int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
    return 0;
}
```

Output:

```
Window: 800 x 600
```

Expression	Meaning / type
this	Pointer to the current object (WindowBox*)
*this	The current object itself (WindowBox lvalue)
return *this;	Returns a reference when the function return type is WindowBox&

Question 5: Private Inheritance Access Violations and Inaccessible Base Cast

Bug locations: Line 13 declares private inheritance; line 24 calls closePort from outside; line 27 attempts to convert SecureChannel* to RawSocket* from outside.

Defect category: Base members become inaccessible through private inheritance; public upcast is blocked.

Explanation: With class SecureChannel : private RawSocket, public and protected members of RawSocket become private members of SecureChannel. They can be used inside SecureChannel's member functions, but not directly by ordinary client code. The conversion from SecureChannel* to RawSocket* is also inaccessible outside the derived class.

Approach A: Add a public using declaration for closePort. This exposes only that selected base member. It does not make public upcasting legal.

Approach B: Use public inheritance if SecureChannel truly is-a RawSocket and clients should be able to use the base interface. Alternatively, use composition if SecureChannel merely has/uses a RawSocket internally.

Corrected code using Approach A:

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened.\n";
    }

    void closePort() {
        std::cout << "Port closed.\n";
    }
};

class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);
        std::cout << "Tunnel encrypted with TLS 1.3\n";
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();

    // RawSocket* socketPtr = &channel; // Still inaccessible
    return 0;
}
```

Output:

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

Important: The using declaration makes closePort callable from main, but it does not expose the RawSocket base conversion. If upcasting is required, change the inheritance to public.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Structured pseudo-code

```
CLASS Person
    ATTRIBUTES:
        personID, fullName, email

    CONSTRUCTOR(id, name, mail)
        personID <- id
        fullName <- name
        email <- mail
END CLASS

CLASS Student VIRTUAL INHERITS Person
    ATTRIBUTES:
        gpa, creditsEarned

    CONSTRUCTOR(id, name, mail, grade, credits)
        CONSTRUCTOR Person(id, name, mail)
        gpa <- grade
        creditsEarned <- credits
END CLASS

CLASS Faculty VIRTUAL INHERITS Person
    ATTRIBUTES:
        academicDepartment, rankTitle

    CONSTRUCTOR(id, name, mail, dept, rank)
        CONSTRUCTOR Person(id, name, mail)
        academicDepartment <- dept
        rankTitle <- rank
END CLASS

CLASS TeachingAssistant INHERITS Student, Faculty
    ATTRIBUTES:
        stipendAmount, assignedCourse

    CONSTRUCTOR(id, name, mail, grade, credits,
                dept, rank, stipend, course)
        CONSTRUCTOR Person(id, name, mail)
        CONSTRUCTOR Student(id, name, mail, grade, credits)
        CONSTRUCTOR Faculty(id, name, mail, dept, rank)
        stipendAmount <- stipend
        assignedCourse <- course

    METHOD DisplayTARecord()
        PRINT personID, fullName, email
        PRINT gpa, creditsEarned
        PRINT academicDepartment, rankTitle
        PRINT stipendAmount, assignedCourse
END CLASS

MAIN
    ta <- NEW TeachingAssistant(
        "P101", "Arun", "arun@example.com",
        8.7, 96, "Information Technology",
        "Assistant Professor", 12000, "C++ Lab")
    ta.DisplayTARecord()
END MAIN
```

Dry run / sample output:

```
Person ID: P101
```

```
Name: Arun
Email: arun@example.com
GPA: 8.7
Credits Earned: 96
Department: Information Technology
Rank: Assistant Professor
Stipend: 12000
Assigned Course: C++ Lab
```

Virtual-base rule: TeachingAssistant is the most-derived class, so it directly initializes the single shared Person base. Student and Faculty do not create duplicate Person subobjects.

Question 2: Polymorphic Payroll with Abstract Base Class and Base Pointers

Structured pseudo-code

```
ABSTRACT CLASS Employee
    PROTECTED:
        employeeID : STRING
        name : STRING

    CONSTRUCTOR(id, employeeName)
        employeeID <- id
        name <- employeeName

    ABSTRACT METHOD CalculatePay() RETURNS REAL

    METHOD DisplayHeader()
        PRINT employeeID, name
END CLASS

CLASS SalariedEmployee EXTENDS Employee
    PRIVATE:
        monthlySalary : REAL

    CONSTRUCTOR(id, name, salary)
        CONSTRUCTOR Employee(id, name)
        monthlySalary <- salary

    METHOD CalculatePay() RETURNS REAL
        RETURN monthlySalary
END CLASS

CLASS HourlyEmployee EXTENDS Employee
    PRIVATE:
        hourlyWage, hoursWorked : REAL

    CONSTRUCTOR(id, name, wage, hours)
        CONSTRUCTOR Employee(id, name)
        hourlyWage <- wage
        hoursWorked <- hours

    METHOD CalculatePay() RETURNS REAL
        regularHours <- MIN(hoursWorked, 40)
        overtimeHours <- MAX(hoursWorked - 40, 0)
        RETURN regularHours * hourlyWage
               + overtimeHours * hourlyWage * 1.5
END CLASS

MAIN
    DECLARE employees AS ARRAY OF POINTER TO Employee
    employees[0] <- NEW SalariedEmployee("E01", "Meena", 8500)
    employees[1] <- NEW HourlyEmployee("E02", "Kumar", 50, 48)

    total <- 0
    FOR EACH employeePtr IN employees
        employeePtr->DisplayHeader()
```

```
        pay <- employeePtr->CalculatePay()
        PRINT pay
        total <- total + pay
    END FOR
    PRINT total

    FOR EACH employeePtr IN employees
        DELETE employeePtr
    END FOR
END MAIN
```

Dry-run calculations:

Employee type	Calculation	Pay
Salaried	Fixed monthly salary	$8,500
Hourly – regular	40 × $50	$2,000
Hourly – overtime	8 × $50 × 1.5	$600
Hourly total	$2,000 + $600	$2,600
Total payroll	$8,500 + $2,600	$11,100

The base class must be abstract because CalculatePay is pure virtual. It should have a virtual destructor in an actual C++ implementation so deleting derived objects through Employee* is safe.

Question 3: Composite Canvas Engine with Member Classes and this Chaining

Structured pseudo-code

```
CLASS Point2D
    PRIVATE:
        x, y : REAL

    CONSTRUCTOR(xValue = 0.0, yValue = 0.0)
        x <- xValue
        y <- yValue

    METHOD Set(xValue, yValue)
        x <- xValue
        y <- yValue

    METHOD GetX() RETURNS REAL
        RETURN x

    METHOD GetY() RETURNS REAL
        RETURN y
END CLASS

CLASS Circle
    PRIVATE:
        centerPoint : Point2D
        radius : REAL

    CONSTRUCTOR()
        centerPoint <- Point2D(0.0, 0.0)
        radius <- 1.0

    METHOD SetCenter(newX, newY) RETURNS REF Circle
        centerPoint.Set(newX, newY)
        RETURN REF THIS

    METHOD SetRadius(r) RETURNS REF Circle
        IF r <= 0
            REPORT "Radius must be positive"
        ELSE
            radius <- r
```

```
        END IF
        RETURN REF THIS

    METHOD Draw()
        PRINT centerPoint.GetX(), centerPoint.GetY(), radius
END CLASS

MAIN
    myCircle <- NEW Circle()
    myCircle.SetCenter(12.5, 30.0)
            .SetRadius(7.5)
            .Draw()
END MAIN
```

Dry run:

Step	Center	Radius	Action
Initial	(0.0, 0.0)	1.0	Circle constructed
SetCenter(12.5, 30.0)	(12.5, 30.0)	1.0	Center updated; returns reference to same Circle
SetRadius(7.5)	(12.5, 30.0)	7.5	Valid radius stored; returns same object
Draw()	(12.5, 30.0)	7.5	Print circle details

Output:

```
Circle center: (12.5, 30.0)
Radius: 7.5
```

Relationship: Circle has a Point2D object (composition / has-a relationship). It is not an inheritance relationship.

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

Structured pseudo-code

```
CLASS Vehicle
    ATTRIBUTES:
        vehicleVIN : STRING
        baseWeightKg : REAL

    CONSTRUCTOR(vin, weight)
        vehicleVIN <- vin
        baseWeightKg <- weight

    METHOD DisplaySpecs()
        PRINT vehicleVIN, baseWeightKg
END CLASS

CLASS AutonomousVehicle EXTENDS Vehicle
    ATTRIBUTES:
        lidarSensorID, firmwareVersion : STRING

    CONSTRUCTOR(vin, weight, lidar, firmware)
        CONSTRUCTOR Vehicle(vin, weight)
        lidarSensorID <- lidar
        firmwareVersion <- firmware
END CLASS

CLASS DeliveryDrone EXTENDS AutonomousVehicle
    ATTRIBUTES:
        payloadCapacityKg, batteryChargePercent : REAL
```

```
    CONSTRUCTOR(vin, weight, lidar, firmware, payload, battery)
        CONSTRUCTOR AutonomousVehicle(vin, weight, lidar, firmware)
        payloadCapacityKg <- payload
        batteryChargePercent <- battery

    METHOD DisplayFlightStatus()
        CALL DisplaySpecs()
        PRINT lidarSensorID, firmwareVersion
        PRINT payloadCapacityKg, batteryChargePercent
END CLASS

MAIN
    dronePtr <- NEW DeliveryDrone(
        "VIN-D100", 8.5, "LDR-09", "v2.4",
        3.0, 87.5)

    vehiclePtr : POINTER TO Vehicle
    vehiclePtr <- dronePtr
    vehiclePtr->DisplaySpecs()

    deliveryPtr <- DOWNCAST vehiclePtr TO DeliveryDrone
    deliveryPtr->DisplayFlightStatus()

    DELETE vehiclePtr
END MAIN
```

Constructor order from root to leaf:

1. Vehicle constructor
2. AutonomousVehicle constructor
3. DeliveryDrone constructor

Dry-run sample output:

```
VIN: VIN-D100
Base weight: 8.5 kg
VIN: VIN-D100
Base weight: 8.5 kg
LiDAR sensor: LDR-09
Firmware: v2.4
Payload capacity: 3 kg
Battery charge: 87.5%
```

C++ implementation note: For safe deletion through Vehicle*, Vehicle should have a virtual destructor. For safe downcasting with dynamic_cast, Vehicle must be polymorphic (for example, declare a virtual destructor or another virtual method).

Question 5: Dynamic Object Pointer Array and Zero-Copy Sorting Engine

Structured pseudo-code

```
CLASS Product
    ATTRIBUTES:
        sku : STRING
        price : REAL
        stockCount : INTEGER

    CONSTRUCTOR(code, cost, stock)
        sku <- code
        price <- cost
        stockCount <- stock

    METHOD PrintProduct()
        PRINT sku, price, stockCount
END CLASS
```

```
MAIN
    N <- 3
    DECLARE catalog AS ARRAY[0..N-1] OF POINTER TO Product

    catalog[0] <- NEW Product("Laptop", 1200, 10)
    catalog[1] <- NEW Product("Mouse", 25, 50)
    catalog[2] <- NEW Product("Monitor", 350, 20)

    FOR i <- 0 TO N - 2
        FOR j <- 0 TO N - i - 2
            IF catalog[j]->price > catalog[j+1]->price
                tempPtr <- catalog[j]
                catalog[j] <- catalog[j+1]
                catalog[j+1] <- tempPtr
            END IF
        END FOR
    END FOR

    FOR i <- 0 TO N - 1
        catalog[i]->PrintProduct()
    END FOR

    FOR i <- 0 TO N - 1
        DELETE catalog[i]
    END FOR
END MAIN
```

Dry-run trace. Initial pointer labels are P1 = Laptop, P2 = Mouse, P3 = Monitor.

Pass / comparison	Prices compared	Swap?	Pointer order after comparison
Initial	—	—	P1(Laptop), P2(Mouse), P3(Monitor)
Pass 1: j=0	1200 > 25	Yes	P2(Mouse), P1(Laptop), P3(Monitor)
Pass 1: j=1	1200 > 350	Yes	P2(Mouse), P3(Monitor), P1(Laptop)
Pass 2: j=0	25 > 350	No	P2(Mouse), P3(Monitor), P1(Laptop)

Sorted output:

```
SKU: Mouse | Price: $25 | Stock: 50
SKU: Monitor | Price: $350 | Stock: 20
SKU: Laptop | Price: $1200 | Stock: 10
```

Why this is zero-copy sorting: The algorithm changes the order of pointer variables in catalog. It does not copy or move the Product objects themselves. Each dynamically allocated Product is deleted exactly once after sorting.

QUICK REVISION – UNIT IV

Topic	Important rule
Diamond problem	Use virtual inheritance on both intermediate paths to share one virtual base.
Virtual base constructor	Initialized by the most-derived class.
Derived constructor	Base constructor is called before derived constructor body; use initializer list.
Base pointer	Can access only members declared in the base's static type.
Downcasting	Use dynamic_cast for checked conversion; source base must be polymorphic.
Object slicing	Copying derived object into base value discards derived-only state.
this / *this	this is a pointer; *this is the current object.
Fluent chaining	Return ClassName& and use return *this;
Private inheritance	Public/protected base interface becomes private in derived class.
Composition	A has-a relationship; contained member objects are constructed with the containing object.
Pointer array sort	Compare with ptr->field; swap pointer variables, not entire objects.
Polymorphic deletion	Use a virtual base-class destructor when deleting derived objects through base pointers.

Exam preparation tip: For debugging answers, write the exact defective line, name the C++ rule, explain why compilation or runtime behavior fails, and then show corrected code and output. For pseudo-code, clearly identify the inheritance hierarchy, pointer/reference types, constructor order, and object ownership.